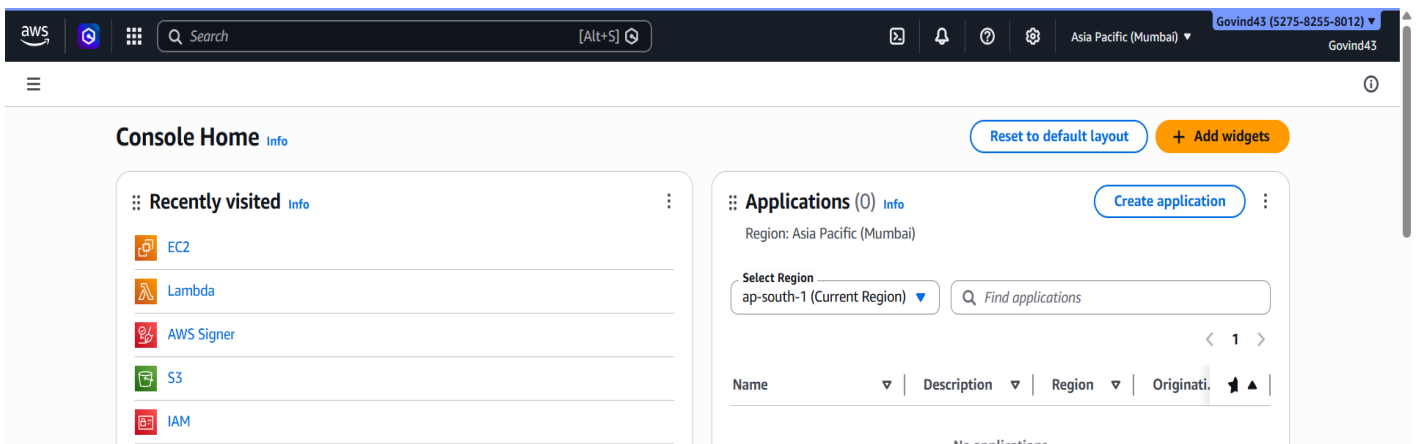
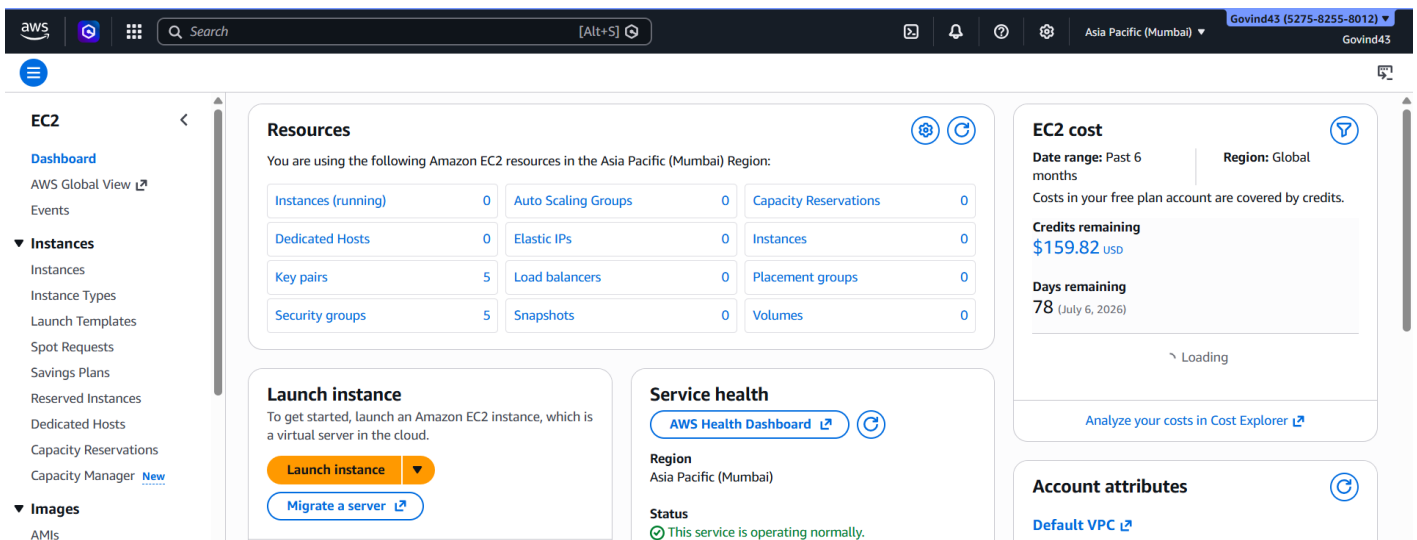


ASSIGNMENT 10 : Deploy and run the project in AWS without using the port.

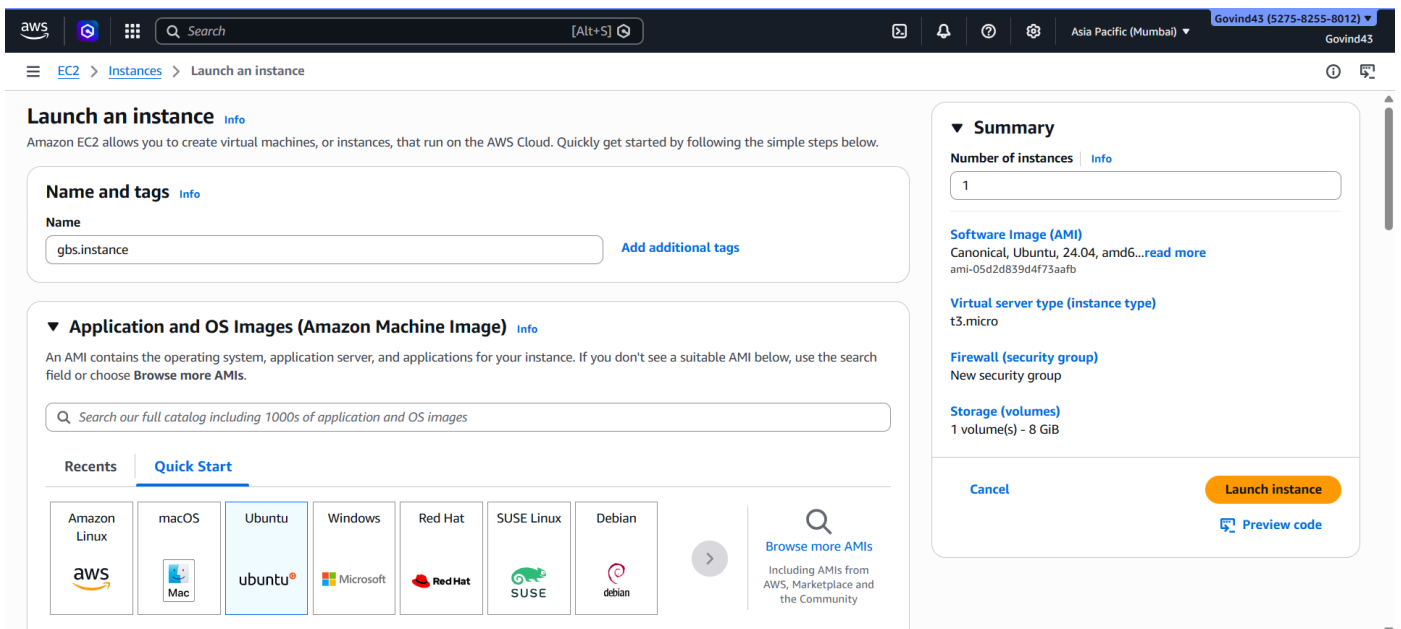
1: Go to the EC2 dashboard.



2. Click on Launch Instance.



3. Give the name of the instance and select UBUNTU as the OS.



4. Select the keypair and select existing security group. (If not created then create the both).

The screenshot shows the 'Launch an instance' page in the AWS Management Console. The 'Key pair (login)' section has 'Gbs.keypair' selected. The 'Network settings' section shows 'vpc-0811d035c1c21c10d' as the VPC, 'No preference' as the subnet, and 'Enable' for auto-assigning a public IP. Under 'Firewall (security groups)', 'Select existing security group' is chosen, and 'security1' is selected from the common security groups list. The 'Summary' panel on the right shows 1 instance, Canonical Ubuntu 24.04 AMI, t3.micro instance type, security1 firewall, and 8 GiB storage. 'Launch instance' and 'Preview code' buttons are visible.

5. In the Advanced settings scroll down and write the code shown in the picture. Then click on “Launch Instance”.

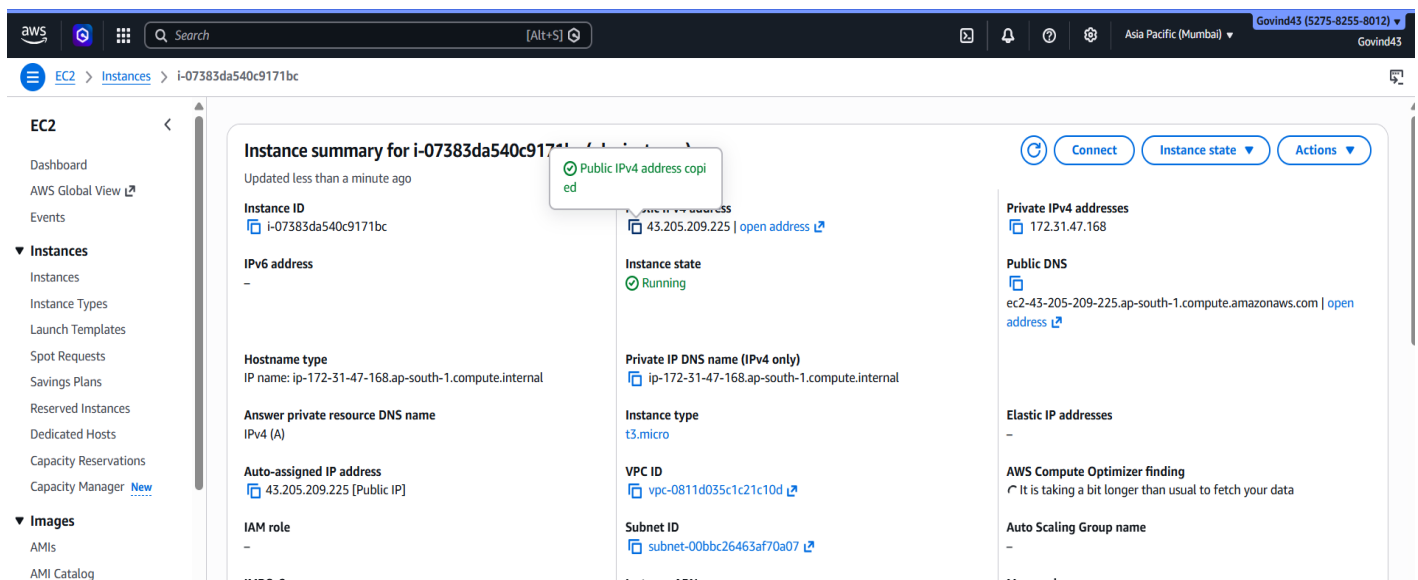
The screenshot shows the 'Advanced settings' section of the 'Launch an instance' page. 'Metadata response hop limit' is set to 2, and 'Allow tags in metadata' is set to 'Select'. In the 'User data - optional' section, a text area contains a bash script for installing Node.js and npm. The 'Summary' panel on the right is identical to the previous step. The 'Launch instance' button is highlighted.

6. The instance is created successfully.

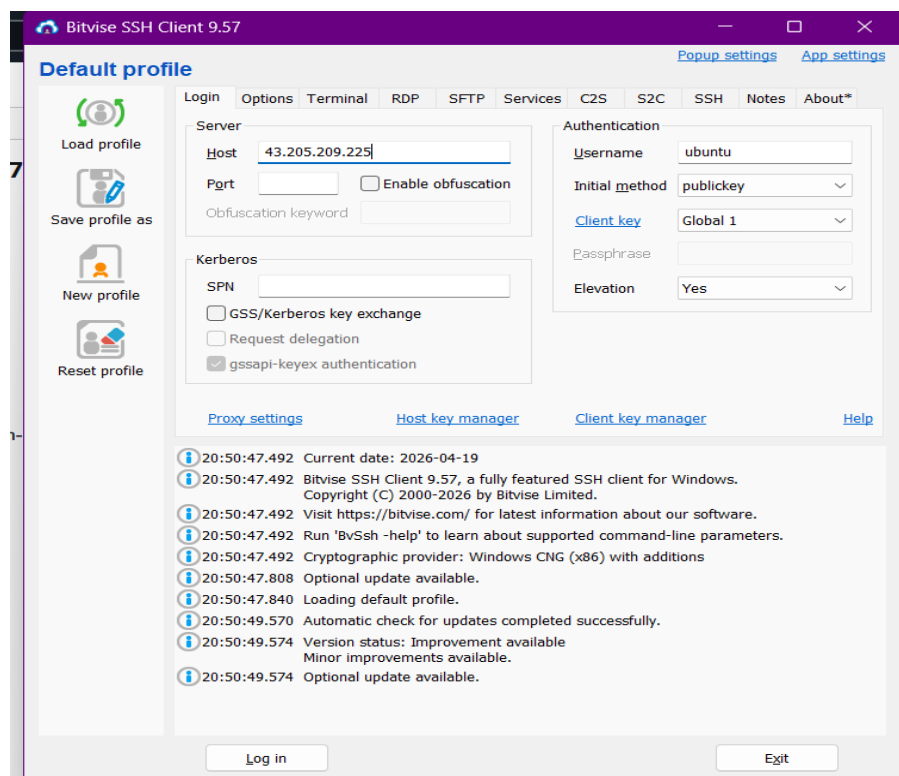
The screenshot shows the 'Instances' page in the AWS Management Console. A table lists the instance 'gbs.instance' with ID 'i-07383da540c9171bc', state 'Running', type 't3.micro', and status 'Initializing'. The instance is in the 'ap-south-1a' availability zone with a public IP of 'ec2-43-205-209-225.ap...'. The 'Launch instances' button is visible in the top right.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4 DNS	Public
gbs.instance	i-07383da540c9171bc	Running	t3.micro	Initializing	View alarms +	ap-south-1a	ec2-43-205-209-225.ap...	43.205

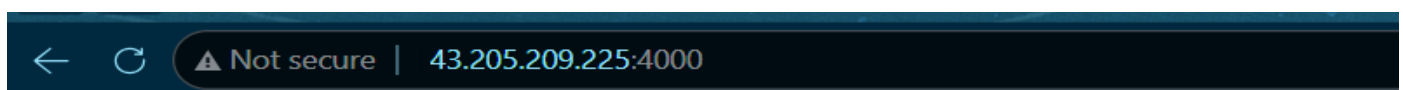
7. Copy paste the Public IPv4 address of the created instance.



8. Open the Bitvise SSH client and paste the ipv4 address here. Under Authentication select username as “ubuntu”, initial method as “public key” , client key as “Global 1” and in client key manager import the keypair selected while creating the instance. Then click on “Log in” then Accept and Save.

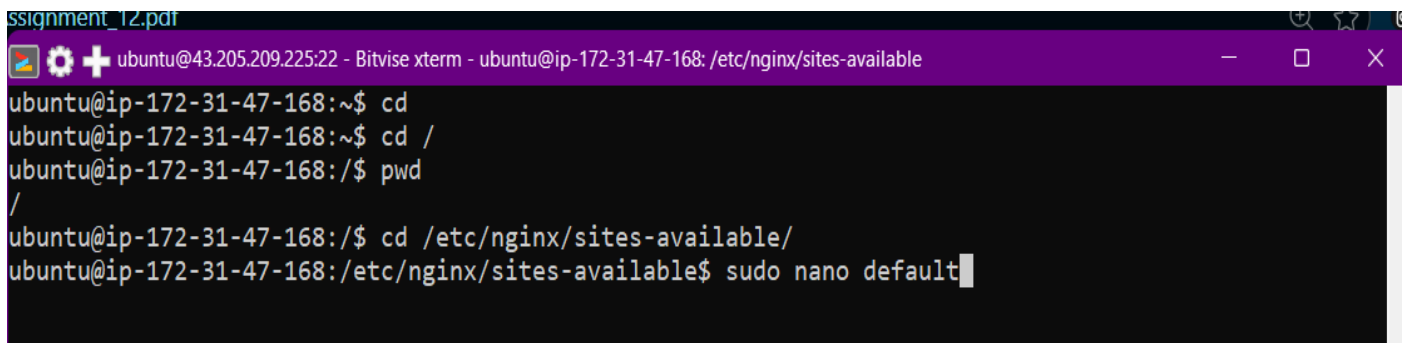


9. Paste the copied ipv4 address on the new tab. Add 4000 (the port number) at the last of the address to see the content of the index.js file.



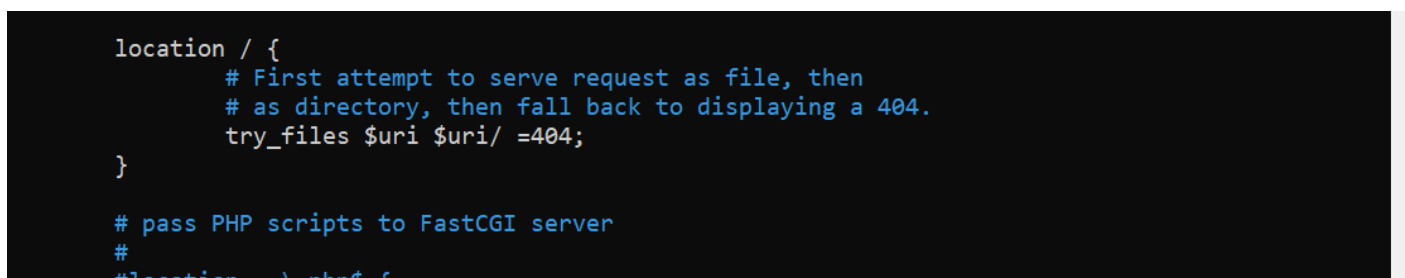
Hello mckv

10. Go to the new terminal window from Bitvise SSH client and write the commands shown in the picture.



```
assignment_12.pdf
ubuntu@43.205.209.225:22 - Bitvise xterm - ubuntu@ip-172-31-47-168: /etc/nginx/sites-available
ubuntu@ip-172-31-47-168:~$ cd
ubuntu@ip-172-31-47-168:~$ cd /
ubuntu@ip-172-31-47-168:/$ pwd
/
ubuntu@ip-172-31-47-168:/$ cd /etc/nginx/sites-available/
ubuntu@ip-172-31-47-168:/etc/nginx/sites-available$ sudo nano default
```

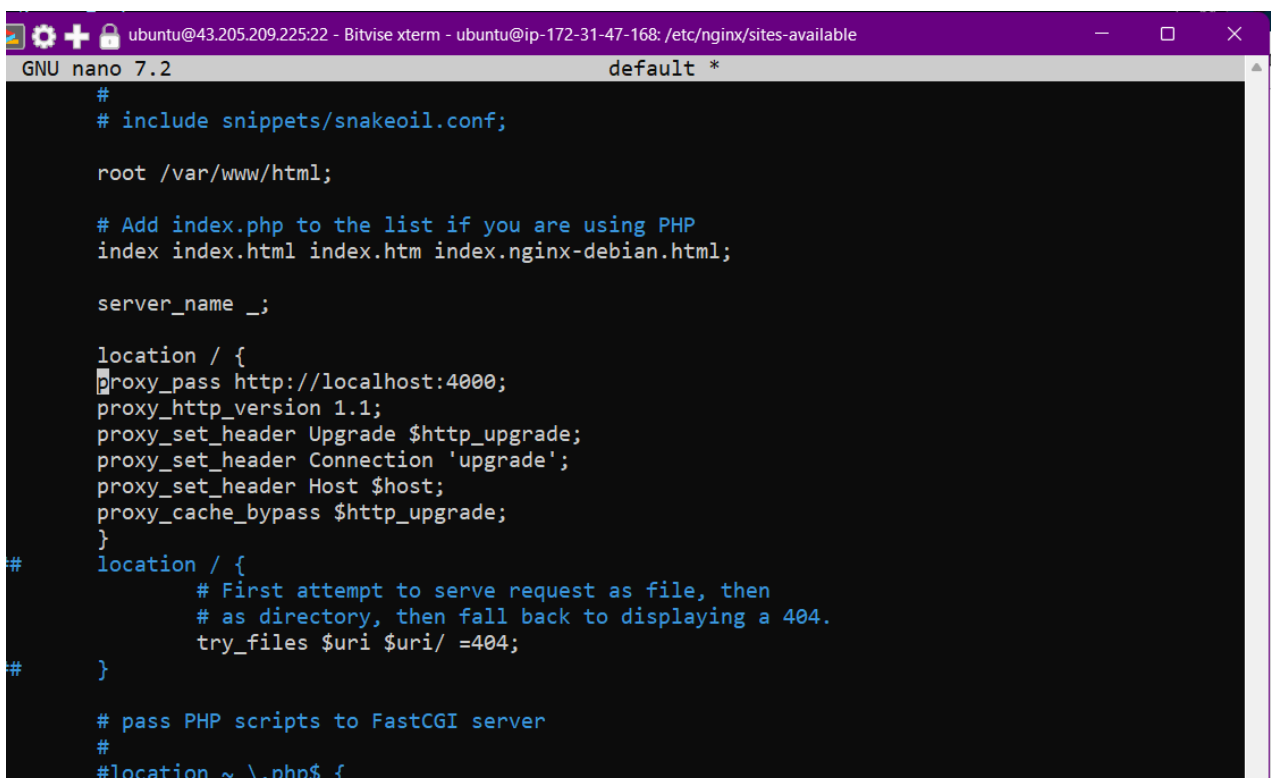
11. Uncomment the code shown in the picture.



```
location / {
    # First attempt to serve request as file, then
    # as directory, then fall back to displaying a 404.
    try_files $uri $uri/ =404;
}

# pass PHP scripts to FastCGI server
#
#location ~ \.php$ {
```

12. Add the following code instead. Then Ctrl + X → Y → Enter. You will come back to bash terminal.



```
GNU nano 7.2 default *
#
# include snippets/snakeoil.conf;

root /var/www/html;

# Add index.php to the list if you are using PHP
index index.html index.htm index.nginx-debian.html;

server_name _;

location / {
    proxy_pass http://localhost:4000;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_cache_bypass $http_upgrade;
}
# location / {
#     # First attempt to serve request as file, then
#     # as directory, then fall back to displaying a 404.
#     try_files $uri $uri/ =404;
# }

# pass PHP scripts to FastCGI server
#
#location ~ \.php$ {
```

13. Write the code → `sudo systemctl restart nginx`
Go back to the created instance in AWS and refresh it.

```
ubuntu@ip-172-31-47-168:/etc/nginx/sites-available$ sudo nano default
ubuntu@ip-172-31-47-168:/etc/nginx/sites-available$ sudo systemctl restart nginx
ubuntu@ip-172-31-47-168:/etc/nginx/sites-available$
```

14. Copy paste the ipv4 address again in the new tab without adding the port number 4000 to it.

You will get your desired output.



Seeing the output we can conclude that our project is successfully deployed and run in AWS without the help of port number.